LAPORAN PRAKTIKUM ALGORITMA DAN PEMROGRAMAN 2

MODUL 12

“SEARCHING”

DISUSUN OLEH:

DYAH IMANSARI

109082500130

S1IF-13-02

DOSEN:

WAHYU ANDI SAPUTRA, S.Pd., M.Eng

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025/2026

DASAR TEORI

Pencarian data merupakan proses menemukan nilai spesifik (seperti nama atau NIM) di dalam sekumpulan data. Berbeda dengan pencarian nilai ekstrem yang pasti ditemukan, pada pencarian spesifik terdapat kemungkinan data yang dicari tidak ada dalam kumpulan data tersebut. Secara umum, terdapat dua metode utama yang sering digunakan:

1. Sequential Search (Pencarian Sekuensial)

Sequential Search atau pencarian linear adalah teknik pencarian yang paling sederhana.

- **Cara Kerja**: Proses dilakukan dengan memeriksa setiap elemen data satu per satu, mulai dari elemen pertama hingga elemen terakhir atau sampai data ditemukan.
- **Karakteristik**:
    - Tidak memerlukan data dalam keadaan terurut.
    - Proses berhenti segera setelah data yang dicari ditemukan (status found menjadi true).
    - Jika pencarian bertujuan menemukan lokasi, algoritma biasanya mengembalikan indeks data tersebut, atau nilai **-1** jika data tidak ditemukan.

2. Binary Search (Pencarian Biner)

Binary Search adalah algoritma pencarian yang jauh lebih efisien untuk menangani data dalam jumlah besar, namun memiliki syarat khusus.

- **Syarat Utama**: Data harus dalam keadaan **terurut** (baik *ascending* maupun *descending*).
- **Cara Kerja**:
    - Algoritma ini menggunakan prinsip *divide and conquer* dengan membagi rentang data menjadi dua bagian (kiri dan kanan).

- Titik tengah (*median*) diambil sebagai pembanding. Jika data yang dicari lebih kecil dari median, pencarian berlanjut ke sisi kiri; jika lebih besar, berlanjut ke sisi kanan.
- Rentang pencarian akan terus menyempit hingga data ditemukan atau variabel batas kiri melewati batas kanan.

3. Pencarian pada Array Tipe Data Struct

Dalam pemrograman (khususnya bahasa Go), pencarian sering dilakukan pada kumpulan data yang kompleks seperti *struct*.

- **Implementasi**: Logika dasarnya tetap sama dengan tipe data sederhana, namun pencarian difokuskan pada salah satu *field* atau kategori tertentu dalam *struct* tersebut (misalnya mencari berdasarkan nim atau nama).
- **Ketentuan**: Jika menggunakan *Binary Search* pada *struct*, data harus sudah terurut berdasarkan *field* yang menjadi kunci pencarian tersebut. Jika data belum terurut berdasarkan kunci yang diinginkan, maka harus menggunakan *Sequential Search* atau mengurutkan data terlebih dahulu.

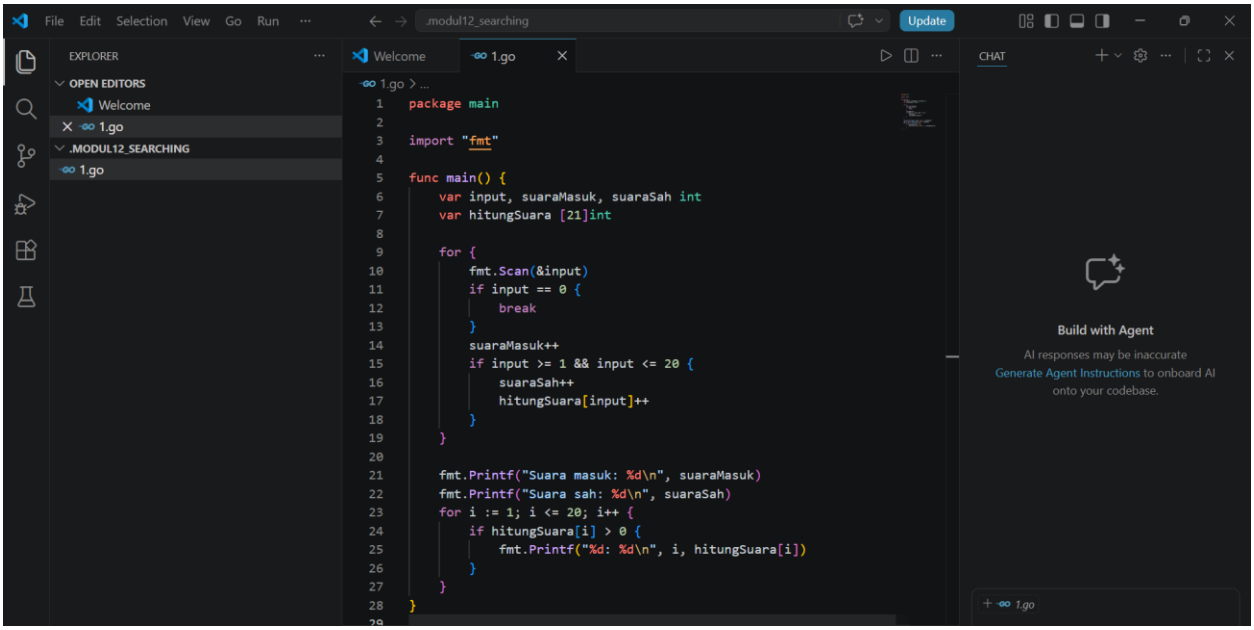
LATIHAN

1. **Latihan 1**

Source Code

```
package main

import "fmt"

func main() {
    var input, suaraMasuk, suaraSah int
    var hitungSuara [21]int

    for {
        fmt.Scan(&input)
        if input == 0 {
            break
        }
        suaraMasuk++
        if input >= 1 && input <= 20 {
            suaraSah++
            hitungSuara[input]++
        }
    }

    fmt.Printf("Suara masuk: %d\n", suaraMasuk)
    fmt.Printf("Suara sah: %d\n", suaraSah)
    for i := 1; i <= 20; i++ {
        if hitungSuara[i] > 0 {
            fmt.Printf("%d: %d\n", i, hitungSuara[i])
        }
    }
}
```

Screenshoot program



Deskripsi program

Program ini mengimplementasikan konsep pengolahan dan pencarian data dalam array. Program membaca setiap nomor pilihan warga satu per satu secara berurutan. Setiap angka yang masuk akan langsung diproses untuk dihitung sebagai total suara masuk. Indeks atau lokasi data sangat penting, jadi program ini menggunakan nomor calon (1–20) sebagai indeks langsung pada array hitungSuara. Jika angka yang dimasukkan valid (1–20), program akan memperbarui nilai pada lokasi tersebut sebagai suara sah. Untuk menampilkan hasil akhir, program melakukan pencarian sekuensial pada array hitungSuara dari indeks 1 sampai 20. Program akan memeriksa setiap elemen, jika ditemukan jumlah suara yang lebih dari 0, maka data calon tersebut akan ditampilkan ke layar.

2. **Latihan 2**

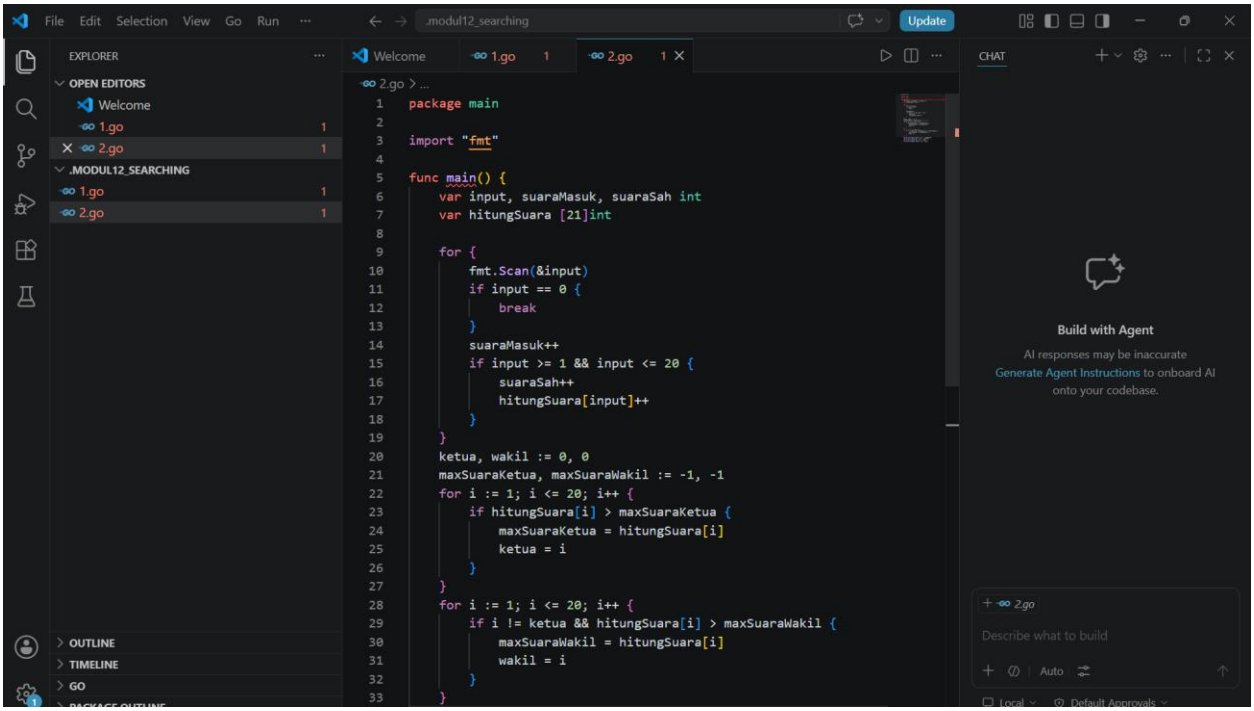
Source Code

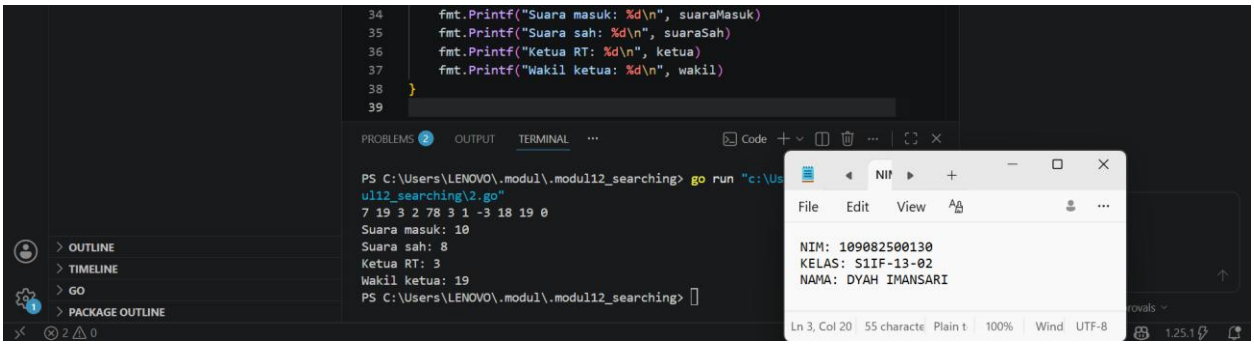
```
package main


import "fmt"
```

```
func main() {
    var input, suaraMasuk, suaraSah int
    var hitungSuara [21]int

    for {
        fmt.Scan(&input)
        if input == 0 {
            break
        }
        suaraMasuk++
        if input >= 1 && input <= 20 {
            suaraSah++
            hitungSuara[input]++
        }
    }
    ketua, wakil := 0, 0
    maxSuaraKetua, maxSuaraWakil := -1, -1
    for i := 1; i <= 20; i++ {
        if hitungSuara[i] > maxSuaraKetua {
            maxSuaraKetua = hitungSuara[i]
            ketua = i
        }
    }
    for i := 1; i <= 20; i++ {
        if i != ketua && hitungSuara[i] > maxSuaraWakil
{
            maxSuaraWakil = hitungSuara[i]
            wakil = i
        }
    }
    fmt.Printf("Suara masuk: %d\n", suaraMasuk)
    fmt.Printf("Suara sah: %d\n", suaraSah)
    fmt.Printf("Ketua RT: %d\n", ketua)
    fmt.Printf("Wakil ketua: %d\n", wakil)
```

```
        }
```

Screenshoot program





Deskripsi program

Program ini merupakan pengembangan dari sistem pemilihan sebelumnya yang kini dilengkapi dengan fitur untuk mencari pemenang (Ketua RT) dan peraih suara terbanyak kedua (Wakil Ketua RT) menggunakan prinsip pencarian nilai ekstrem. Setelah seluruh data suara dibaca secara berurutan dan divalidasi ke dalam array hitungSuara sesuai indeks nomor calon 1–20, program melakukan penelusuran sekuensial untuk membandingkan jumlah suara setiap calon guna menentukan siapa yang memiliki suara tertinggi. Jika terdapat beberapa calon dengan jumlah suara yang sama, program akan secara otomatis memilih calon dengan nomor peserta terkecil sebagai pemenang karena penelusuran dilakukan berurutan dari indeks terkecil. Logika pencarian ini memastikan bahwa posisi Ketua dan Wakil Ketua diisi oleh calon yang paling tepat berdasarkan kriteria jumlah suara

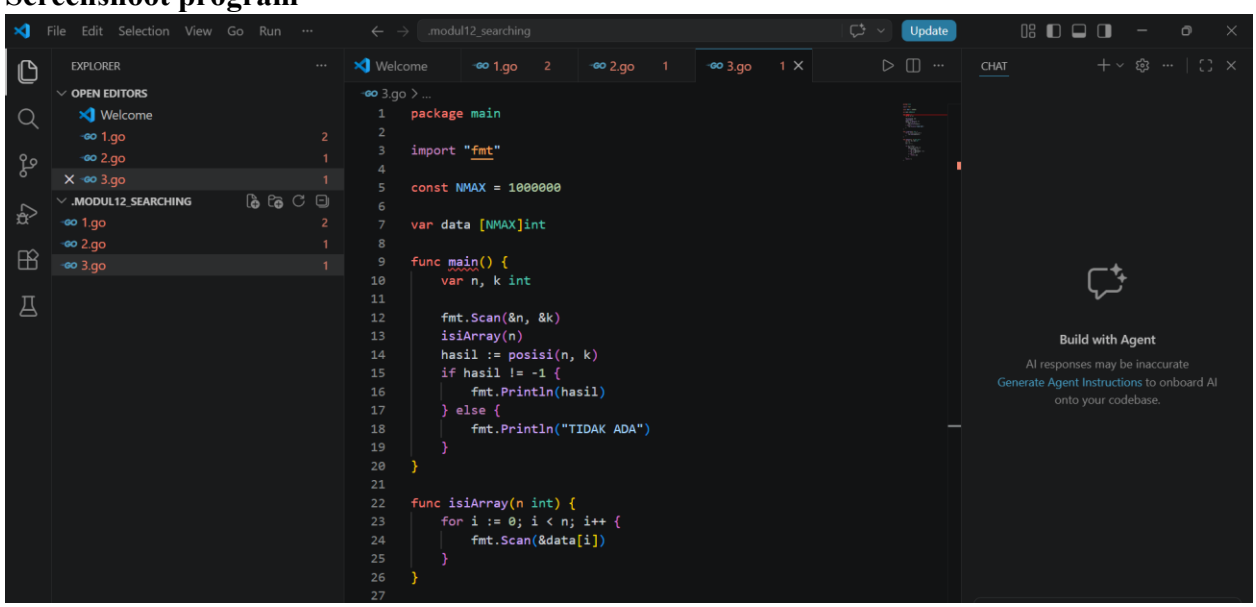
dan urutan nomor peserta, sementara data yang tidak valid tetap diabaikan melalui proses penyaringan nilai di luar rentang 1–20.

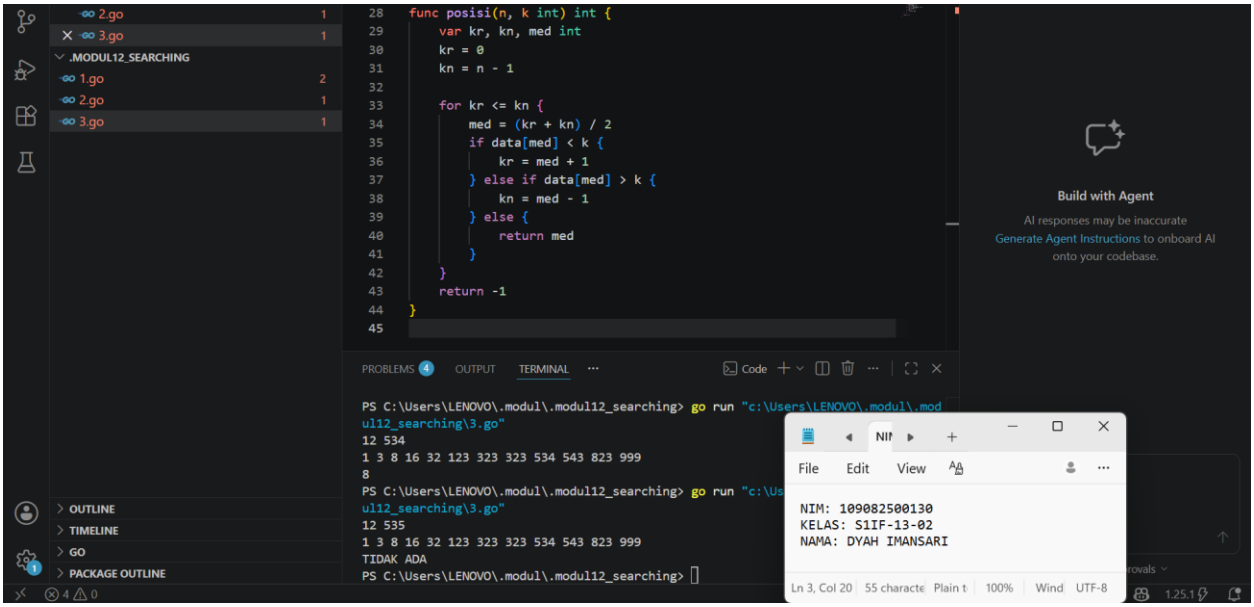
3. **Latihan 3**
   Source Code

```
package main

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func main() {
    var n, k int

    fmt.Scan(&n, &k)
    isiArray(n)
    hasil := posisi(n, k)
    if hasil != -1 {
        fmt.Println(hasil)
    } else {
        fmt.Println("TIDAK ADA")
    }
}

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    var kr, kn, med int
    kr = 0
```

```
        kn = n - 1

        for kr <= kn {
            med = (kr + kn) / 2
            if data[med] < k {
                kr = med + 1
            } else if data[med] > k {
                kn = med - 1
            } else {
                return med
            }
        }
        return -1
    }
```

Screenshoot program





Deskripsi program

Program ini mengimplementasikan algoritma Binary Search untuk menemukan posisi suatu bilangan dalam kumpulan data yang sudah terurut membesar. Program menggunakan subprogram berupa prosedur isiArray untuk memasukkan n buah data ke dalam array dan fungsi posisi untuk melakukan pencarian nilai k secara efisien dengan membagi rentang data menjadi dua bagian terus-menerus. Jika nilai k ditemukan, fungsi akan mengembalikan indeksnya (dimulai dari 0), namun jika tidak ditemukan setelah rentang pencarian habis, program akan memberikan keluaran "TIDAK ADA”.

DAFTAR PUSTAKA

Opraza, A. (2023, September 6). *Memahami Algoritma Pencarian Binary Search*. Arsy Opraza. Diakses pada 14 Mei 2026, dari https://dasarpemrogramangolang.novalagung.com/A-array.html

Tim Penyusun. (2026). *Modul Praktikum Algoritma dan Pemrograman 2*. Bandung: Fakultas Informatika, Telkom University.

Universitas Telkom Jakarta. (2026, Februari 21). *Searching Algorithm: Linear Search dan Binary Search*. Universitas Telkom Jakarta. Diakses pada 14 Mei 2026, dari https://jakarta.telkomuniversity.ac.id/searching-algorithm-linear-search-dan-binary-search/